Prompt Unificado — Corrigir e Blindar Agente de IA (Railway + Supabase + Gemini)

Cole este prompt diretamente na CLI do Gemini:

Preciso que você corrija e melhore meu agente de IA para resolver todos os problemas de performance, conexão e confiabilidade. Meu stack é: Python + Gemini + Supabase (PostgreSQL) + Railway. Aplique as seguintes correções no meu código:

BLOCO 1 — Corrigir erro SSL SYSCALL / Connection timed out (Supabase)

1. Use asyncpg para gerenciar o pool de conexões com PostgreSQL/Supabase.
2. Configure o pool com: min_size=1, max_size=10, max_inactive_connection_lifetime=300, command_timeout=30.
3. Adicione TCP keepalive nas configurações do servidor: tcp_keepalives_idle=60, tcp_keepalives_interval=10, tcp_keepalives_count=5.
4. Sempre que ocorrer erro com as palavras “ssl syscall”, “connection timed out”, “connection reset”, “broken pipe”, “could not receive data” ou “consuming input”, feche o pool atual e recrie automaticamente antes de tentar novamente.
5. Implemente uma função de keepalive que executa SELECT 1 no banco a cada 5 minutos para manter a conexão viva.
6. Use ssl=“require” na conexão pois o Supabase exige SSL.

BLOCO 2 — Corrigir lentidão e falta de resposta (Gemini)

1. Envolva todas as chamadas ao Gemini em asyncio.wait_for() com timeout=90 segundos.
2. Implemente retry automático com 3 tentativas e backoff exponencial: aguarde 2s, 4s e 8s entre tentativas.
3. Use stream=True nas chamadas ao Gemini para que a resposta chegue em chunks progressivos.

4. Use gemini-2.5-flash como modelo padrão (é 3x mais rápido que o Pro). Deixe configurável via variável de ambiente GEMINI_MODEL.

BLOCO 3 — Nunca deixar o usuário sem resposta

1. Ao receber qualquer mensagem, envie IMEDIATAMENTE "⏳ Processando sua mensagem…" antes de chamar qualquer API.
2. Edite essa mensagem com a resposta real quando estiver pronta.
3. Se usar streaming, edite a mensagem a cada 0.5 segundos com o texto acumulado + cursor "▌ " no final.
4. Se após todas as tentativas ainda houver erro, envie: "⚠ Não consegui processar agora. Tente novamente em instantes." — nunca deixe o usuário sem resposta.

BLOCO 4 — Múltiplos usuários simultâneos

1. Implemente asyncio.Queue() com 5 workers paralelos para processar múltiplos usuários sem travar.
2. Mantenha histórico de conversa por usuário em um dicionário {user_id: historico}.
3. Limite o histórico a 20 mensagens por usuário para não estourar o contexto do modelo.

BLOCO 5 — Evitar hibernação no Railway

1. Adicione uma função assíncrona que faz GET na rota /health do próprio servidor a cada 8 minutos (Railway hiberna após 10 minutos de inatividade no plano gratuito).
2. Crie a rota GET /health que retorna {"status": "ok"} para ser usada pelo keepalive.
3. Use a variável de ambiente RAILWAY_PUBLIC_DOMAIN para montar a URL do ping automaticamente.

BLOCO 6 — Boas práticas gerais

1. Adicione logging detalhado em cada etapa: recebimento, tentativa, tempo de resposta, confirmação de entrega.
2. Use variáveis de ambiente para todas as chaves e configurações: GEMINI_API_KEY, DATABASE_URL, RAILWAY_PUBLIC_DOMAIN, GEMINI_MODEL.

3. Crie uma função inicializar_sistema() que inicializa banco, agente e keepalives de forma ordenada no startup da aplicação.
4. Adicione requirements.txt com: google-generativeai, asyncpg, aiohttp, fastapi, uvicorn, python-dotenv.

Aplique tudo isso mantendo a lógica e funcionalidades existentes do meu agente. Use Python com asyncio. Não remova nenhuma funcionalidade atual, apenas adicione as correções e melhorias.